OASIS: Repairing Stale and Correlation-Blind Query-Optimizer Statistics from Query Feedback

Qichu Tian[a,*], Heng Chen[a]

[a] *Xi'an Jiaotong University, Xi'an, China*

Abstract

Cost-based optimizers plan with column statistics that drift stale between refreshes and that assume columns are independent; both errors mislead the planner. Query execution already exposes them—for every filter predicate the executor reveals the selectivity the optimizer mis-estimated—so we ask how far query feedback alone, without rescanning the table, can maintain optimizer-facing statistics. We separate two regimes.

For a *single* column, feedback determines the marginal up to a small free part, and a maximum-entropy projection (the classical ISOMER step) is already near-optimal: we give an identifiability condition for when feedback pins the marginal, and show on real datasets, against self-tuning histograms and a learned prior, that nothing beats the projection there—single-column repair is easy. The optimizer's larger residual error is the *independence assumption* on correlated columns, and there feedback helps decisively: a feedback-driven joint repair cuts conjunctive-predicate Q-error by up to 31% (13.6% in aggregate) over independence on real correlated columns, the gain concentrated where the columns are correlated.

We package this as OASIS, a statistics-layer middleware that maintains optimizer statistics from query feedback between full `ANALYZE` refreshes. Injected into a real PostgreSQL planner, feedback-maintained single-column statistics cut row Q-error from 27.8 to 2.4 and lift fresh-plan match from 56% to 96% without a table scan, and the maintained marginals stay safe for composition and join estimators to consume.

*Corresponding author

Email addresses: `mizukiqwq@stu.xjtu.edu.cn` (Qichu Tian), `hengchen@xjtu.edu.cn` (Heng Chen)

Keywords: Cardinality estimation, Query optimization, Self-tuning histograms, Independence assumption, Query feedback, Statistics maintenance

1. Introduction

Cost-based optimizers (CBOs) choose query plans from estimates of predicate selectivity, intermediate cardinality, and operator cost [1, 2]. In many analytical database systems these estimates rely on column-level statistics such as histograms, most-common-value lists, and density summaries [3, 4, 5, 6], refreshed periodically by `ANALYZE`-style procedures that read a full or sampled view of the table [7]. This maintenance strategy is simple and robust, but it leaves a persistent staleness gap: tables keep receiving inserts, deletes, and updates while the optimizer plans with the last statistics snapshot. As the gap widens, stale histograms induce systematic selectivity errors that propagate through the cost model into poor plans [8, 9, 10, 11]. A second, orthogonal error is structural rather than temporal: per-column statistics carry no cross-column information, so the optimizer falls back on the attribute-value-independence assumption and systematically mis-estimates conjunctions over correlated columns [4, 8].

A DBMS already obtains the signal to repair both errors while executing ordinary queries. For a filter predicate, the optimizer's estimated and the executor's observed selectivity expose a local discrepancy between the stale statistics and the current data; for a conjunction over several columns, the same feedback reveals the joint selectivity the independence assumption got wrong. Feedback-driven self-tuning histograms turn such observations into statistics updates—STHoles, SASH, and ISOMER are the canonical examples [12, 13, 14, 15]—and ISOMER makes the principle explicit: among all distributions consistent with the observed feedback, choose the *maximum-entropy* one—the least-committed completion of the part feedback leaves undetermined.

This paper asks a deployment question: *how far can query feedback alone, with no table rescan, maintain the statistics a cost-based optimizer actually plans with?* The answer splits cleanly into two regimes, and that split is the organizing idea.

Single columns are easy. An equi-depth histogram with B buckets has only $B - 1$ free interior boundaries; a feedback window pins the cumulative distribution at the observed boundaries and leaves a small free part. We

make this precise with an identifiability condition (Section 4) for when feedback determines the marginal, and we find empirically—on real datasets, against self-tuning histograms and against a prior trained directly on the optimizer's error—that once the feedback projection is applied, *nothing beats the maximum-entropy completion*: a learned model gives no edge over the classical ISOMER step. Single-column repair therefore needs no machine learning; the principled projection suffices, and we prove and measure when it does.

Independence is the bottleneck. The optimizer's larger residual error is the independence assumption introduced above. Maximum entropy over a single column cannot touch it, but conjunctive feedback can—it directly observes the joint mass independence gets wrong. A feedback-driven joint repair (a two-dimensional max-entropy projection onto observed conjunctive masses) cuts conjunctive-predicate Q-error by up to 31%—13.6% in aggregate—over the independence assumption on real correlated columns, with the gain concentrated where the columns are correlated (Section 6.2). This is where feedback maintenance earns its keep for the optimizer.

The resulting system is OASIS, a statistics-layer middleware that maintains optimizer statistics from query feedback between full `ANALYZE` refreshes, with no base-table rescan. A *conversion layer* maps native engine statistics to a canonical distribution view and back (Section 3.2); a *repair layer* applies the feedback-consistency projection—to a single marginal, and, for correlated columns, to the joint (Sections 3.3, 6.2); and a *safety layer* routes among candidate corrections on a deployment-visible signal so an imperfect correction never degrades below the maximum-entropy default (Section 3.3). Because the correction is inserted at the statistics layer, the optimizer's search and cost model are untouched. This design yields four contributions:

- An *identifiability* analysis of feedback-driven repair: a rank condition (Proposition 1) for when query feedback pins the optimizer-relevant statistic, and when it leaves a free part no consistent method can resolve from feedback alone (Sections 2.3, 4).
- A clean *dichotomy*, validated on real data: single-column repair is easy—the maximum-entropy projection is near-optimal, and a learned prior and the self-tuning histograms STHoles and QuickSel-H give no edge over it (Section 6.1)—whereas the independence assumption is the dominant residual optimizer error.
- A *feedback-driven independence repair* that beats the attribute-value-independence assumption by up to 31% (13.6% aggregate) on real

correlated columns, with the gain concentrated where the columns are correlated (Section 6.2).

- *OASIS*, a no-rescan statistics-maintenance middleware validated in a real PostgreSQL planner—feedback-maintained single-column statistics cut row Q-error from 27.8 to 2.4 and lift fresh-plan match from 56% to 96%—with the maintained marginals shown safe for composition and join estimators to consume and a residual-gated router that never degrades below the maximum-entropy default (Section 6.3).

2. Background and Problem Formulation

2.1. Feedback-Based Statistics Maintenance

Because an `ANALYZE`-style refresh must read a full or sampled view of the table, refreshes are scheduled rather than continuous; between them the table keeps changing while the optimizer plans from the last snapshot, so a once-accurate histogram drifts from the current distribution—especially around recent values, range predicates, and localized updates.

Query execution already exposes this drift: for each filter predicate, the optimizer's estimated selectivity and the executor's observed selectivity reveal a local disagreement between the old statistics and the current data. Feedback-driven self-tuning histograms use such observations to repair statistics: STHoles splits and refines buckets from feedback [13]; ISOMER selects the maximum-entropy histogram among those consistent with the feedback and drops old constraints when drift makes the active set inconsistent [15]; QuickSel-H fits a query-driven selectivity model on the same no-rescan interface [16]. These methods all take query feedback as input and emit a corrected single-column statistic without touching the base data. Their key difference is how they complete the parts of the distribution that feedback does not determine. OASIS keeps the same interface and the same maximum-entropy completion for a single column; its leverage is extending the feedback projection to the cross-column joint and routing among candidates for safety.

2.2. Statistics and Objective

We model a column histogram as an equi-depth quantile summary with B buckets. Its interior boundaries $v_1, \ldots, v_{B-1}$ correspond to fixed quantile levels $p_1, \ldots, p_{B-1}$ and induce a piecewise-linear CDF used to estimate range-predicate selectivity. Native DBMS statistics may also include most-common-value lists, density vectors, or null fractions; OASIS operates on a canonical

one-dimensional CDF view and converts to and from native statistics through an engine-specific adapter (Section 3.2).

Let H_0 be the stale histogram from the last refresh and H^* the current post-drift distribution. For a predicate σ, the optimizer derives an estimate $\hat{s}_{H_0}(\sigma)$ from H_0, while the executor can later reveal an actual selectivity $s^*(\sigma)$. A feedback window $\mathcal{O} = \{o_1, \ldots, o_K\}$ records such observations; each o_j contains a predicate type, a boundary, an estimated selectivity, and an actual selectivity. We measure error with Q-error,

$$\mathrm{QErr}(\hat{s}, s^*) = \max\left(\frac{\hat{s}}{s^*}, \frac{s^*}{\hat{s}}\right). \tag{1}$$

For one predicate, Q-error is a scalar. To compare two statistics objects, we average this scalar over future predicates. Let $\sigma \sim \mathcal{D}_{\mathrm{fut}}$ denote a future predicate drawn from the workload distribution, and let s^*_σ be its true selectivity under the current distribution H^*. The goal is to construct corrected statistics $\hat{H}$ that lower this average future-predicate error,

$$\mathbb{E}_{\sigma\sim\mathcal{D}_{\mathrm{fut}}}[\mathrm{QErr}(\mathrm{CDF}_{\hat{H}}(\sigma), s^*_\sigma)] < \mathbb{E}_{\sigma\sim\mathcal{D}_{\mathrm{fut}}}[\mathrm{QErr}(\mathrm{CDF}_{H_0}(\sigma), s^*_\sigma)]\,.$$

A deployable repair must derive $\hat{H}$ from H_0 and feedback only, avoid table rescans, run near planning time, and degrade gracefully when feedback is sparse.

2.3. Degrees of Freedom in Feedback-Based Repair

Feedback repair is underdetermined for an elementary reason. A feedback window reports the true mass on a handful of intervals; between and beyond those intervals, the data can be arranged in many ways and still reproduce every observation. The repaired marginal is thus *pinned* where feedback falls and *free* everywhere else, and that free part (Figure 1) is what any repair method must fill. We make this precise and measure how large the free part is below.

To make this precise, we rewrite the feedback constraints in a representation that makes them *exactly linear*. Take the active feedback predicates together with the stale histogram grid; their boundary points—the grid points and the distinct feedback endpoints—partition the domain into C cells. We then represent the marginal by its vector of cell masses $m \in \Delta$. Since these C masses are nonnegative and sum to one, the simplex Δ has dimension $C - 1$.

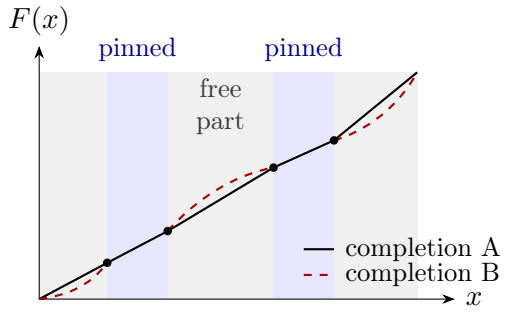


Figure 1: Why feedback repair is underdetermined. A feedback window pins the cumulative mass F at the predicate boundaries it observes (dots); on the pinned intervals (blue) every consistent completion agrees, but on the regions the feedback leaves free (grey)—the free part—completions may differ (solid versus dashed) while still matching every observation—so the free regions are exactly what any repair method must fill, and once feedback pins them no choice remains (Section 4).

In this *cell-mass representation*, each feedback observation constrains only the total mass on its predicate interval. Because every predicate endpoint is a cell boundary, that interval mass is an exact sum of cell masses (two such sums for a `BETWEEN`). The feedback window therefore imposes a system of linear equality constraints $Am = y$, with $A \in \mathbb{R}^{K' \times C}$. Let $r = \text{rank}(A)$. The marginals consistent with the feedback form the affine slice $\{m \in \Delta : Am = y\}$, whose free dimension is $(C - 1) - r$. We call this the *free subspace* that the feedback leaves.

A deployed histogram exposes only $B-1$ of these C cell boundaries, and on a fixed grid the two representations are interchangeable—inverting the implied CDF recovers the cell masses from the exposed quantiles. Section 4 therefore measures an *operational* rank on those $B - 1$ boundaries—a deployable proxy for rank(A)—and turns this view into the pinned-regime result—when feedback pins the projected marginal, and when a free part remains for any prior to fill.

This formulation is deliberately single-column. OASIS does not infer multi-column dependence from single-column feedback, and it does not repair join-key dependence, physical design, or cost-model calibration. The single-column marginal is nonetheless a load-bearing input: copula, IPF, and factorized-join estimators all factor the joint into one-dimensional marginals and a dependence structure, so correcting the marginal propagates to any consumer that composes it (Sections 6.3.1 and 6.3.2 hold the dependence

structure fixed and vary only the marginal). Where cross-column correlation or join-key skew is the dominant *residual* error, an external dependence model remains necessary and OASIS is complementary to it—an assumption we state rather than measure for any one production system.

3. OASIS Design

3.1. System Overview

OASIS is a middleware that sits at the statistics layer, between the statistics store and the optimizer, and is organized as three layers (Figure 2). The *conversion layer* (L1, Section 3.2) maps the engine's native statistics object to a canonical distribution view and back, so the rest of the system is engine-independent. The *repair layer* (L2) applies the feedback-consistency projection that maintains the statistic—to a single marginal (Section 3.3) and, for correlated columns, to the joint (Section 6.2); it is training-free—no model and no offline training (we include a learned prior only as a baseline and find it gives no edge, Section 6.1). The *safety layer* (L3, Section 3.3) routes among candidate corrections on a deployment-visible signal so an imperfect correction never degrades below the maximum-entropy default.

When the optimizer requests a histogram, the request passes down through the three layers and back (Figure 2). Two integration points suffice: a query-completion listener that records predicate-level feedback from executor row counts, and a statistics-assembly hook that substitutes the corrected histogram before it reaches the optimizer. Because the correction is inserted at the statistics layer, the optimizer's search procedure and cost model are untouched, and feedback collection adds only bounded per-conjunct bookkeeping.

OASIS names the approach—feedback-consistency repair guarded by a safety router—and in its default deployed form applies the hard projection, with the residual-gated Router (detailed in Section 3.3) as the fallback. The external baselines are the stale histogram and ISOMER (the maximum-entropy projection initialized from stale), together with the self-tuning histograms STHoles and QuickSel-H.

3.2. Statistics-Format Conversion

Many engines do not expose a standalone editable equi-depth histogram. PostgreSQL, for example, couples histogram boundaries with MCV lists and other per-column summaries. We instantiate the interface for PostgreSQL-style statistics with a three-phase adapter: reconstruct a canonical distribution

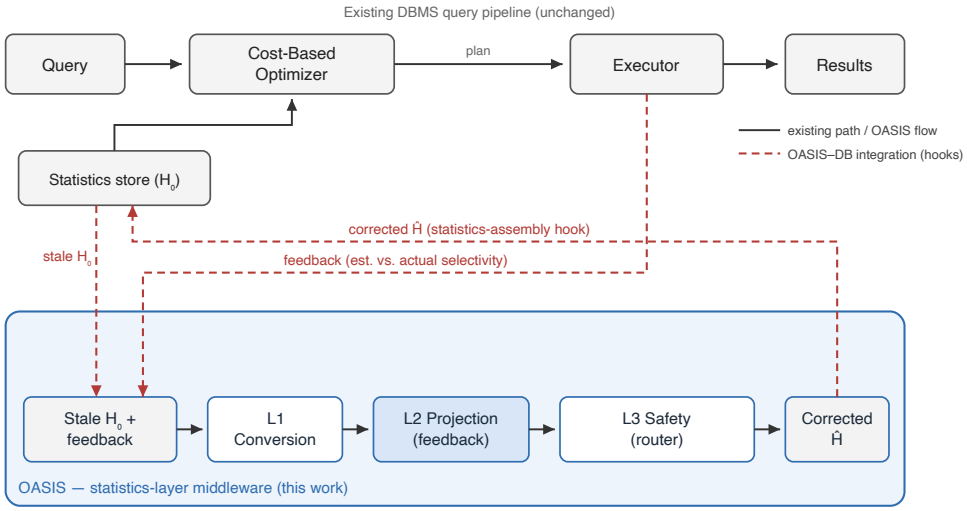


Figure 2: OASIS as a statistics-layer middleware. The existing DBMS query pipeline (solid, top) is left unchanged. OASIS runs its own input-to-correction flow (solid): it reads the stale histogram H_0 and a recent feedback window, then applies conversion (L1), feedback projection (L2), and safety routing (L3) to emit a corrected histogram $\hat{H}$. It attaches to the pipeline only through the dashed integration hooks—tapping predicate feedback from the executor and substituting $\hat{H}$ into the statistics the optimizer reads. All three layers are deterministic; L2 is the training-free feedback projection.

view from the native object, apply correction on the canonical view, and decompose the corrected view back into native fields. The round trip preserves total mass to machine precision (residual-quantile MAE mean 2.4×10^{-17}, max 9.2×10^{-17}), keeps selectivity estimates consistent (global MAE 1.1×10^{-4}, worst case below 0.39%), and completes in 0.066–0.073 ms.

Only the conversion layer is engine-specific; the projection and routing layers operate on the canonical CDF and are unchanged across systems. Porting OASIS to a new engine therefore reduces to writing one adapter. The effort depends on how the engine exposes single-column statistics. MySQL and MariaDB keep equi-height/equi-width histograms in the data dictionary and expose them through `ANALYZE TABLE ... UPDATE HISTOGRAM` and `information_schema`, which maps to the canonical CDF almost directly. Microsoft SQL Server stores step-based histograms readable via `DBCC SHOW_STATISTICS`, again a close fit. Oracle couples height-balanced and hybrid histograms with column-level density and is closest to the PostgreSQL

case we implement, where the adapter must also reconcile a most-common-value list. Engines that do not expose an editable histogram object would require correcting statistics through their maintenance API rather than in place. We validate the PostgreSQL adapter end-to-end; the others are design targets, not claims, but they share the same canonical interface.

3.3. The Repair Layer: Feedback Projection and the Router

The repair layer projects an input histogram onto the active feedback so the result is consistent with what the executor observed, and a light router guards the deployed correction. The same operator serves the single marginal and, in two dimensions, the cross-column joint (Section 6.2).

The hard operator is the ISOMER/IPF feedback-consistency projection [15]: it builds the interval partition induced by the active feedback predicates, initializes cell masses from the input histogram (the stale marginal, or the independence grid for a joint), and performs cyclic I-projections until every active predicate's interval mass matches its observed selectivity. When sequential drift makes the active set inconsistent, the oldest constraints are dropped first. The operator is not new; its role here is to realize the picture of Section 2.3—it pins the statistic exactly on the feedback span and leaves the maximum-entropy default elsewhere—and to provide the safe form for joins and the planner.

On future single-column predicates, matching a handful of recent intervals exactly can overfit; a soft operator that minimizes $\mathrm{KL}(p \,\|\, p_{\text{prior}}) + \lambda \sum_j w_j (A_j p - y_j)^2$ over the same partition generalizes slightly better, with $w_j = \exp(-(\text{conflict}_j/\tau)^2)$ down-weighting feedback the most recent observations contradict. We keep it as a router candidate but do not build on it.

For robustness the deployed system can route among candidates—stale, the hard projection (ISOMER), and the soft projection—selecting, per case, the one with the lowest *in-window feedback residual*: the mean absolute discrepancy, over the full recent observation window, between a candidate's estimate and the observed selectivity. The gate never observes test predicates, true cardinalities, runtimes, or plan shapes, and because the pool contains the hard projection it can only lower the in-window residual. It is a fallback that reverts to the maximum-entropy projection when a candidate over-extrapolates, not a plan-shape guarantee; a plan-shape-aware gate is left to future work (Section 8). On the planner configurations it selects ISOMER throughout (Section 6.3.3).

4. Identifiability: When Feedback Pins the Statistic

Section 2.3 framed feedback repair as constraining a distribution the observations leave only partly determined. This section makes the *identifiability* question precise: when does query feedback pin the optimizer-relevant statistic, and when does it leave a free part that no feedback-consistent method can resolve from feedback alone? We answer with a rank condition and quantify how the free dimension shrinks as feedback accumulates.

We now state the pinned regime precisely.

Proposition 1 (Pinned regime)**.** *Adopt the cell-mass representation of Section 2.3: the histogram grid and the distinct active feedback endpoints partition the domain into C cells, the marginal is the cell-mass vector $m \in \Delta = \{m \in \mathbb{R}^C : m \geq 0,\ \mathbf{1}^\top m = 1\}$ with $\dim \Delta = C - 1$, and the active feedback is* consistent and noise-free*, written $Am = y$ with $A \in \mathbb{R}^{K' \times C}$ and $r = \operatorname{rank}(A)$, the normalization $\mathbf{1}^\top m = 1$ being independent of the rows of A. Let Π be the I-projection onto the feasible set $F = \{m \in \Delta : Am = y\}$, applied identically to every method's prior, and let $N = \ker A \cap \ker \mathbf{1}^\top$, so $\dim N = (C-1) - r$. Then:*

1. *for any two priors p, p' the projected marginals satisfy $\Pi(p) - \Pi(p') \in N$, so two priors can differ in their projected output* only *on the $((C-1)-r)$-dimensional free subspace N; and*
2. *if $r = C - 1$ (the feedback identifies the marginal in this representation) then $N = \{0\}$ and F is a single point, so $\Pi(p) = \Pi(p')$ for* every *pair of priors. In particular the projections of the stale histogram (ISOMER) and of any other prior return the identical marginal.*

Proof sketch. The I-projection onto a nonempty closed convex set returns a point of that set, so $\Pi(p), \Pi(p') \in F$ and both satisfy $Am = y$ and $\mathbf{1}^\top m = 1$. Their difference $d = \Pi(p) - \Pi(p')$ therefore obeys $Ad = 0$ and $\mathbf{1}^\top d = 0$, i.e. $d \in N$, which proves (i). For the dimension, N is the intersection of $\ker A$ (codimension r) with the normalization hyperplane $\ker \mathbf{1}^\top$ (codimension 1), and the two are independent by assumption, so $\dim N = C - 1 - r$. When $r = C - 1$ we get $\dim N = 0$; the feasible set F is nonempty by consistency, hence a single point onto which every prior projects, which proves (ii). □

Two qualifications fix the scope of Proposition 1. First, the coincidence in (ii) concerns *any prior closed by the same projection* Π, not the native STHoles or QuickSel-H algorithms, which need not reduce to this I-projection;

Table 1: Feedback-constraint rank and residual free degrees of freedom (DOF) of the marginal versus feedback budget K. We report an *operational* rank on the $B-1=9$ exposed histogram boundaries—a deployable proxy for the exact cell-level rank(A) (Section 2.3). Free DOF is the free-subspace dimension the projection leaves for the prior to complete; once feedback saturates the DOF the marginal is pinned and every method applying the same projection coincides (the pinned regime).

K	mean rank	mean free DOF	marginals pinned
2	2.32	6.68	0%
4	4.65	4.35	0%
6	6.95	2.05	6%
8	8.71	0.29	73%
12	8.98	0.02	98%
16	8.98	0.02	98%

our evaluation makes the comparison like-for-like by applying the same operator to every prior (Section 6.1), the setting in which the coincidence holds. Second, the rank r above is the *exact* cell-level rank(A), whereas Table 1 reports an *operational* rank on the $B-1$ exposed histogram boundaries (Section 2.3)—a deployable proxy for r, not r itself.

The Proposition is an *identifiability* statement: when the feedback is rank-deficient ($r < C-1$) a free part N remains, and *no* feedback-consistent method can resolve it from feedback alone—every such method must commit to some default there. ISOMER commits to maximum entropy [15], the least-committed choice; STHoles and QuickSel-H commit to their own structural assumptions. Whether *anything* can beat the maximum-entropy default inside N is then an empirical question, and the answer depends sharply on dimension: for a single column it cannot (Section 6.1), whereas for the cross-column joint it can, because conjunctive feedback supplies the joint information the single-column free part never sees (Section 6.2).

We now make this prediction quantitative, since it governs how every later result should be read. An equi-depth marginal with B=10 buckets exposes $B-1=9$ free interior boundaries. For each held-out case we count the independent feedback constraints (distinct interior boundary anchors, merged within tolerance) and the residual free degrees of freedom (DOF). Table 1 reports the mean rank and free DOF as a function of the feedback budget K.

The free DOF collapses from 6.68 at K=2 to ≈ 0 once the feedback constraints saturate the $B-1$ boundaries—here by $K \geq 12$ on this constraint-

spanning predicate population, with 73% of marginals already pinned at K=8. The free part is therefore a property of *which regions of the domain the feedback constrains*, not merely of how many observations there are: a finite window pins the marginal near the observed predicates and leaves it free elsewhere. Two consequences follow. (i) *At saturation* the marginal is pinned and projection-based methods coincide (Proposition 1); the dense PostgreSQL planner regime (Section 6.3.3) is exactly this tie. (ii) *Below saturation* a free part remains, but—as Section 6.1 shows on real data—for a single column the maximum-entropy default already fills it near-optimally, so the residual free dimension is not an opportunity any prior can exploit. The exploitable structure lies one level up, in the cross-column joint (Section 6.2).

5. Experimental Methodology

Our evaluation follows a maintained statistic along its deployment path, asking at each level the one question that level can answer. We first establish the *single-column* object on real data: feedback projection repairs the stale marginal, and the maximum-entropy completion is near-optimal—a learned prior and the self-tuning histograms give no edge (Section 6.1). We then move one level up, to the *cross-column* joint, where feedback repairs the independence assumption that per-column statistics structurally cannot (Section 6.2). Finally we hand the corrected statistics to real downstream consumers—composition estimators, a join kernel, and a live PostgreSQL planner—and ask whether they stay safe (Section 6.3). The evaluation is deliberately *planner-facing*: we report the corrected statistics object together with the row estimates and plan shapes a real optimizer derives from it, and use runtime only as a single narrow TPC-H reality check that calibrated statistics track fresh-statistics behavior, not as a claim of wall-clock superiority over stale statistics.

Unless stated otherwise, every feedback-driven method consumes the same observation budget K=16 and emits corrected quantiles on the same B=10 grid; where a drift intensity q is reported (Sections 6.3.1, 6.3.2) it scales how far the post-drift distribution moves from the snapshot. The baselines fall into three classes: (i) *true self-tuning histograms*—STHoles [13] and ISOMER [15], run as their published feedback-driven algorithms; (ii) an *adapted histogram-interface baseline*, QuickSel-H [16], exposing QuickSel's density estimate on the shared grid (a faithful interface adaptation, not a line-by-line reproduction); and (iii) an *internal learned query-driven control*,

Table 2: Evaluation metrics. Direction marks whether lower (↓), higher (↑), or near-one ($\to 1$) is better. $\hat{s}, s^*$ are estimated and actual selectivity; p is the corrected marginal; $A_j p$ is its mass on feedback interval j with observed value y_j.

Metric	Definition	Dir.
Q-error	$\max(\hat{s}/s^*,\ s^*/\hat{s})$, Eq. (1)	↓
Selectivity MAE	mean $\lvert\hat{s} - s^*\rvert$ over predicates	↓
Quantile MAE	mean $\lvert v_i - v_i^{\text{fresh}}\rvert$ over boundaries	↓
Feedback residual	mean $\lvert A_j p - y_j\rvert$ on the observation window	↓
Row Q-error	Q-error of `EXPLAIN` estimated rows vs `COUNT(*)`	↓
Fresh-plan match	frac. of queries whose plan equals the fresh-stats plan	↑
Recovery	frac. of stale/fresh plan disagreements the method fixes	↑
New deviations	frac. of stale/fresh agreements the method breaks	↓
Join regret	selected-plan proxy cost / optimal proxy cost	↓
Time/Fresh	geomean warm-cache runtime ratio to fresh stats	$\to 1$

LQM, a monotone sigmoid-basis network fit online to the same feedback window, in the style of learned query-driven estimators [17, 18] but restricted to single-column repair with the same no-rescan inputs. QuickSel-H and LQM are matched-interface stand-ins, not the published systems—they calibrate where a flexible feedback-only fit lands relative to the projection, and we draw no general claim against learned cardinality estimators. In the result tables the *OASIS* column is the deployed feedback projection, *Unproj. prior* the same correction *without* that projection (a control), and the residual-gated selector appears as *Hybrid* or *Router* (the same selector).

Every method starts from the *same* stale statistics, sees the *same* feedback window, is projected onto the *same* constraints, and is evaluated on the *same* held-out split; no method receives oracle access, and "fresh" is computed from the post-drift data only as an accuracy upper bound. Table 2 defines the metrics used throughout.

Drift model and its justification. We do not have access to a production stream of statistics-refresh histories, so we drive staleness with controlled drift, chosen to mimic how analytical tables actually evolve between `ANALYZE` passes. The drift intensity q scales how far the post-drift distribution moves from the snapshot H_0 (in units of inter-quantile shift), so larger q corresponds to a longer or more turbulent refresh interval; we sweep $q \in \{1, \ldots, 30\}$ to cover mild to severe staleness. The drift *families* (batch loads, range and date-band

Table 3: Single-column repair on held-out *real* datasets (Power, Forest, Census), reusing the prior trained only on a disjoint pool of real columns. Stale is the uncorrected histogram; ISOMER is the maximum-entropy feedback projection; STHoles and QuickSel-H are self-tuning histograms; the learned prior is reported routed. All feedback methods cluster tightly and the learned prior gives *no* gain over the max-entropy projection: on real single-column data, feedback projection is already near-optimal. Geometric-mean future-predicate Q-error (↓).

Dataset	K	Stale	ISOMER	STHoles	QuickSel-H	Learned
Power	16	1.511	1.182	1.171	1.245	1.164
	2	1.528	1.362	1.356	1.408	1.346
Forest	16	1.766	1.125	1.113	1.104	1.104
	2	1.813	1.472	1.464	1.470	1.451
Census	16	1.038	1.020	1.030	1.030	1.020
	2	1.040	1.039	1.050	1.066	1.041

shifts, skew evolution, outlier bursts, multimodal and seasonal mixtures) are the update patterns that workload-evolution and learned-estimator drift studies report as the practical causes of estimator degradation [19, 20, 11, 21]. We treat the drift model as a stated, transparent assumption rather than a claim about any single production system.

6. Empirical Evaluation

6.1. Single-Column Repair: the Projection Is Near-Optimal on Real Data

The decisive single-column test is on real data. We draw individual columns from three standard datasets (Power, Forest/Covtype, Census), drive drift with a sliding window over the column in file order, and repair each column from a feedback window of K observations. The learned prior is trained *only* on a disjoint pool of real columns (Wine Quality, Bike Sharing) and never sees the test datasets. Every method initializes the *same* feedback projection; all are scored on held-out future predicates.

Two things stand out in Table 3. First, feedback repair works: the maximum-entropy projection (ISOMER) cuts stale Q-error sharply—Forest from 1.77 to 1.13, Power from 1.51 to 1.18—confirming that query feedback maintains a single-column statistic well with no table rescan. Second, and central to this paper's design, *no method beats the projection*: STHoles,

QuickSel-H, and the learned prior all land within 1–2% of ISOMER, at both dense (K=16) and sparse (K=2) feedback. The maximum-entropy completion is already near-optimal for a single column, exactly as the identifiability analysis (Section 4) leads one to expect: the free part a single column leaves is small and carries little structure a learned model can exploit beyond the least-committed default. *Single-column repair therefore needs no machine learning*; we adopt the training-free projection and turn, in Section 6.2, to the cross-column setting where feedback genuinely helps.

6.2. Repairing the Independence Assumption from Feedback

Single-column repair is the *easy* regime: once the feedback projection is applied, the maximum-entropy completion is near-optimal and a learned prior adds nothing (Section 6.1). The optimizer's larger error is structural—the attribute-value-independence (AVI) assumption, under which a conjunction's selectivity is the product of the per-column selectivities. For correlated columns this is wrong no matter how accurate each marginal is [4, 8], and no amount of single-column repair can fix it. Query feedback, however, observes exactly the quantity AVI gets wrong: the executor returns the true selectivity of a *conjunctive* predicate.

We test whether conjunctive feedback can repair the joint. For a pair of columns we form the true joint on a $G \times G$ grid (G=12); the AVI baseline is the outer product of the *true* marginals—the best case for independence, which isolates the correlation error from any marginal error. The feedback estimate is a two-dimensional max-entropy (IPF) projection that starts from the AVI grid and rescales it to match K=16 observed conjunctive rectangle masses—the natural two-dimensional analogue of the single-column ISOMER projection. Both are scored on held-out conjunctive rectangle predicates by geometric-mean Q-error. We use eight real correlated column pairs drawn from the same public datasets as the rest of the evaluation (Wine Quality, Bike Sharing, Forest/Covtype, Census, Power), spanning weak to strong correlation.

Table 4 reports the result, and it follows the structure the analysis predicts. Where the columns are essentially independent (Census `age`/`hours`, r=0.07) feedback repair barely moves the estimate (+3.9%): there is no correlation to recover and AVI is already right. Where the columns are correlated, feedback repair wins decisively—from +7% to +31% across the correlated pairs, largest on the strongly dependent `casual`/`cnt` pair, for a 13.6% aggregate reduction

Table 4: Feedback-driven joint repair versus the independence assumption (AVI) on real correlated column pairs. AVI uses the true marginals (best case for independence); the feedback estimate is a 2D max-entropy (IPF) projection onto K=16 observed conjunctive rectangle masses. Geometric-mean conjunctive-predicate Q-error on held-out predicates; the gain over AVI is concentrated on the correlated pairs (negligible at $r \approx 0$).

Real column pair	Pearson r	AVI	Feedback	Improv.
`census:age/hours`	+0.07	1.186	1.140	+3.9%
`forest:elev/roadway`	+0.37	1.326	1.237	+6.8%
`wred:density/alcohol`	−0.50	1.419	1.264	+10.9%
`wwhite:freeSO2/totSO2`	+0.62	1.491	1.312	+12.0%
`wred:fixedAcid/pH`	−0.68	1.532	1.358	+11.3%
`bike:casual/cnt`	+0.69	2.088	1.436	+31.2%
`forest:hill9/hill3`	−0.78	1.822	1.475	+19.1%
`bike:temp/atemp`	+0.99	1.688	1.492	+11.6%
Aggregate		1.538	1.328	+13.6%

in conjunctive Q-error over AVI. The gain follows the *presence* of correlation—vanishing as columns approach independence—rather than scaling strictly with $|r|$. Unlike the single-column case, the cross-column information that independence discards is large and *feedback-fillable*: conjunctive observations carry exactly the joint mass per-column statistics structurally lack. This is where feedback maintenance earns its keep for the optimizer—and, as in the single-column case, the repair operator is the training-free max-entropy projection, now in two dimensions.

Validation on the Join Order Benchmark. The pairs above use real columns but a controlled predicate suite. We therefore repeat the test on the Join Order Benchmark (JOB), the standard real-data cardinality-estimation benchmark, querying its IMDB tables inside a live PostgreSQL 14 planner. For correlated column pairs we score the geometric-mean Q-error of conjunctive predicates (range on numeric columns, equality on categorical ones) under four estimators: PostgreSQL's default (per-column statistics plus independence); PostgreSQL's own multi-column *extended* statistics (`CREATE STATISTICS`, built from a full scan); independence with the *true* marginals (AVI, which isolates the cross-column error from per-column error); and the OASIS feedback-repaired joint (K=16 observed conjunctive masses, no scan).

Table 5 shows three things. First, where the optimizer's error is genuinely

Table 5: Cross-column repair on the Join Order Benchmark (real IMDB tables, live PostgreSQL 14). Geometric-mean conjunctive-predicate Q-error (↓) for PostgreSQL-default (per-column + independence), PostgreSQL extended statistics (`CREATE STATISTICS`, scan-built), independence with true marginals (AVI), and the OASIS feedback-repaired joint (no scan). The conservative cross-column effect is AVI→OASIS; `person_info` is a near-independent control.

Real IMDB column pair	r	PG-def.	PG-ext.	AVI	OASIS
`title.prod_year/kind`	0.35	5.69	5.63	5.70	**1.95**
`aka_title.prod_year/kind`	0.29	3.85	3.93	3.75	**1.67**
`cast_info.nr_order/role`	0.01	78.2	77.6	2.27	**1.63**
`movie_comp.ctype/cid`	0.51	62.4	62.3	1.25	**1.09**
`title.season/episode`	−0.02	37.7	39.4	1.51	**1.45**
`person_info.itype/pid` (ctrl)	0.04	58.9	58.9	1.08	1.09

the independence assumption—`production_year`×`kind_id` in `title` and `aka_title`, where PostgreSQL's default already matches the true-marginal AVI baseline—feedback joint repair cuts conjunctive Q-error by 56–66%. Second, on a near-independent control (`person_info`, an identifier column) it has no effect (−1%), confirming the repair is specific to genuine dependence. Third, PostgreSQL's conjunctive estimates on these columns are badly off (up to 78×), and its own extended statistics do *not* help—their MCV/dependency machinery covers common combinations, not the correlated tail—whereas the feedback-repaired joint brings the error to 1.1–2.0×. Where PostgreSQL's error far exceeds the true-marginal baseline (e.g. `cast_info`, 78× versus 2.3×), most of that gap is PostgreSQL's *per-column* estimation error on skewed columns rather than independence; we therefore report the conservative cross-column effect as the AVI→OASIS reduction, which is feedback repair alone.

6.3. Downstream Optimizer Consumption

A corrected single-column statistic is only as good as what its consumers can safely do with it. This section tests a single claim—*a feedback-consistent (projected) statistic is safe for any consumer, whereas an unprojected correction can be useless or actively harmful*—and tests it along an escalating axis rather than re-establishing it four times. Each rung adds exactly one thing the previous cannot: six composition estimators show the claim is *consumer-agnostic* (Section 6.3.1); a bilinear FactorJoin kernel is the *nonlinear stress*

case, where an unprojected correction turns harmful (Section 6.3.2); a live PostgreSQL planner is the *real-system* test on row estimates and plan shapes (Section 6.3.3); and TPC-H is the lone *runtime* reality check (Section 6.3.4). Because the OASIS and *Unproj. prior* columns differ only in the projection, every rung also isolates how much of the benefit is the projection rather than the prior. The primary evidence is estimate quality and plan *shape*; runtime enters only in the final, deliberately narrow check.

6.3.1. Composition-Family Generalization

We embed OASIS as a drop-in marginal source in six independently designed composition estimators—the independence/maximum-entropy estimator, IPF/Sinkhorn on a 2D grid, and Gaussian, Clayton, Gumbel, and Frank copulas—varying only the marginal input. The dependence structure is held fixed and identical for every marginal, so this is a clean marginal ablation, and the Archimedean copulas are intentionally mis-specified for the Gaussian-correlated data.

Figure 3a shows the claim holds across all six estimators: the feedback projection recovers 19.7%–36.0% of the stale joint Q-error, while an unprojected correction yields weak or negative gains. The benefit is therefore *consumer-agnostic*—it does not depend on a single dependence model and survives even the intentionally mis-specified Archimedean copulas.

6.3.2. FactorJoin Integration

To test propagation into a learned *join* estimator, we embed OASIS in the bin-based join kernel of FactorJoin [22]. For an equi-join $A.k = B.k$ it estimates $\sum_{\text{bin}} \text{cnt}_A[\text{bin}]\,\text{cnt}_B[\text{bin}]/\text{dv}[\text{bin}]$, where each per-bin key frequency comes from a single-column key histogram, which we make the OASIS-correctable object. The keys are generated independently, so only the single-column key histogram varies across methods.

Figure 3b is the sharp case the escalation was built for. Because the kernel is bilinear in per-bin mass, it amplifies mass-concentration errors, and the unprojected correction becomes *actively harmful*—aggregate join Q-error 1.621, worse than the stale 1.213 it was meant to repair. Feedback projection reverses this: OASIS improves over stale at every drift level (+12.0%–+23.2%, +15.3% aggregate; the Hybrid +16.1%) and tracks the fresh marginal throughout. A correction that helped the mild consumers above can *hurt* a nonlinear one unless it is first projected onto the feedback.

Table 6: Multi-configuration PostgreSQL planner-only statistics-injection evidence. Values are means over configurations; parentheses show one standard deviation across configurations. True rows use `COUNT(*)`; estimated rows and plan shapes use `EXPLAIN (FORMAT JSON)`. No query runtime is measured.

Method	Row QE	Fresh Plan	Recovery	New Deviations
Stale	28.453 (6.576)	56.1% (5.1)	0.0% (0.0)	0.0% (0.0)
ISOMER	2.390 (0.286)	96.0% (1.9)	91.9% (3.9)	1.0% (1.0)
Unproj. prior	3.894 (1.252)	90.9% (5.1)	80.7% (14.4)	2.4% (1.9)
OASIS	2.377 (0.269)	95.9% (1.9)	91.7% (3.8)	1.0% (1.0)
Soft	2.452 (0.312)	96.0% (1.9)	91.9% (3.9)	1.0% (1.0)
Router	2.386 (0.288)	96.1% (2.0)	92.1% (4.1)	1.0% (1.0)
Fresh	1.415 (0.092)	100.0% (0.0)	100.0% (0.0)	0.0% (0.0)

6.3.3. PostgreSQL Planner-Only Statistics Injection

We validate that corrected statistics affect a real optimizer pipeline. The experiment builds alternative single-column statistics for `fact.x` and injects them into `pg_statistic`; true rows come from `COUNT(*)`, estimated rows and plan shapes from `EXPLAIN (FORMAT JSON)`. No query runtime is measured. The batch covers two drift directions, two table sizes, and three seeds (12 configurations).

Table 6 is the strongest DBMS-facing evidence, and its claim is deliberately one of *plan-shape safety*, not accuracy superiority over ISOMER. Pooling over all query instances, stale statistics give aggregate row Q-error 27.768 and match the fresh plan on 56% of queries. (All numbers in this paragraph are pooled over query instances; Table 6 instead reports the per-configuration mean±sd, which differs slightly—e.g. stale row Q-error is 28.453 averaged over configurations versus 27.768 pooled.) An unprojected correction reduces row Q-error to 3.703 but introduces new plan deviations in 2.3% of cases. The feedback projection (OASIS) reduces row Q-error to 2.362, matches the fresh plan on 95.9% of queries, recovers 92.1% of stale/fresh disagreements, and cuts new deviations to 1.1%. Crucially, OASIS (2.362) and ISOMER (2.374) are near-identical here—the rank mechanism, not a disappointment: these planner-facing windows are dense enough to pin the active constraints (Section 4), so hard projection converges to nearly the same marginal whichever prior seeds it, and the Router selects ISOMER on all twelve configurations. On a real optimizer, then, feedback maintenance restores near-fresh plans with no table scan, and the projection alone suffices.

Table 7: TPC-H SF10 DML-drift study, aggregated over three independent refresh-stream seeds (mean $\pm$ std; six curated date-sensitive queries each; planner-only statistics injection, PostgreSQL 14). The pretrained OASIS prior is reused with no TPC-H retraining. *Scan Q-err* is the row-estimation error on the drifting date column. *Time/Fresh* and *Time/Stale* are geometric-mean ratios of warm-cache `EXPLAIN ANALYZE` execution time to the fresh- and stale-statistics runs. Calibrated statistics (OASIS, Router) track fresh statistics within the reported distribution (median Time/Fresh ≈ 1, worst case ≤ 1.40) and induce no plan-shape deviation from fresh across the 18 query instances. The stale wall-clock baseline is not a reliable target: a stale mis-estimate can pick a coincidentally cheaper-to-run plan (Time/Stale > 1 here), whereas in other configurations it picks a much slower one; we therefore make no runtime-superiority claim and report that OASIS reproduces fresh-statistics behavior.

Method	Scan Q-err	Time/Fresh	Time/Stale
Stale	5.23 ± 0.24	0.65 ± 0.07	1.00 ± 0.00
ISOMER	2.02 ± 0.08	0.98 ± 0.07	1.51 ± 0.07
Unproj. prior	2.66 ± 0.07	1.02 ± 0.02	1.59 ± 0.18
OASIS	2.02 ± 0.10	0.96 ± 0.04	1.49 ± 0.11
Router	2.01 ± 0.10	1.01 ± 0.02	1.57 ± 0.15
Fresh	2.01 ± 0.07	1.00 ± 0.00	1.56 ± 0.17

6.3.4. TPC-H DML-Drift Runtime Sanity Check

This runtime check states its claim up front: calibrated statistics *reproduce fresh-statistics behavior*—accuracy, plan shape, and runtime—rather than beat the stale baseline on wall-clock. We load TPC-H SF10, drift it with an RF1-style insert stream that shifts `lineitem.l_shipdate` and `orders.o_orderdate` into a new date band, and inject single-column statistics for those columns exactly as above, reusing the *same* pretrained prior with no TPC-H retraining (also a cross-schema generalization test). For six date-sensitive queries we record scan row Q-error, plan shape, and warm-cache `EXPLAIN ANALYZE` time over three refresh-stream seeds. Because warm-cache runtime is determined by the induced plan, we time each distinct (query, plan) once at full repetition and assign it to every method sharing that plan.

Table 7 reports the result. Accuracy is unambiguous: stale scan Q-error 5.23 ± 0.24 is repaired to 2.02 for OASIS and 2.01 for the Router, matching fresh (2.01), on a standard schema with no retraining. Calibrated statistics track the *fresh*-statistics runtime to within a few percent (time-to-fresh 0.96 for OASIS, 1.01 for the Router) because they induce the fresh plans. The *stale* wall-clock baseline is not a reliable target: here the stale mis-estimate

happens to pick a cheaper-to-run plan, so calibrated and fresh statistics run $\approx 1.5\times$ slower than stale—the price of the correct plan, paid equally by fresh—whereas in a larger-drift configuration the same mis-estimate produced a plan $1.56\times$ slower than fresh. The deployment-relevant invariant is that calibrated statistics track fresh statistics in both accuracy and runtime, while stale statistics make the wall-clock outcome unpredictable in either direction. This is a single-schema check on six queries; we draw no broader runtime-superiority conclusion.

Because a median or geomean can hide a bad tail, we report the runtime distribution against the deployment target, fresh statistics, over all 18 query instances (six queries, three seeds). Calibrated statistics induce the fresh plan on every instance—*zero* plan deviations from fresh for OASIS, the Router, and ISOMER—so there is no plan-shape tail. The runtime-to-fresh ratio stays tight: median 0.96 and worst-case 1.23 for OASIS, median 1.00 and worst-case 1.40 for the Router. The only large ratio appears against *stale*, on the single query (TPC-H Q14) whose plan changes stale→fresh: under stale's coincidentally cheap plan it runs much faster—up to $\approx 13\times$ in the worst instance—but fresh statistics show a comparable gap there, so this is the cost of switching to the correct plan, not a regression introduced by calibration. No calibrated method is slower than fresh by more than $1.4\times$ on any query or seed.

6.4. Overhead and Deployment

A per-column correction is the feedback projection (a handful of I-projection sweeps over the B buckets) plus the PostgreSQL-style format conversion (0.066–0.073 ms); it completes in well under a millisecond and uses no learned model. Feedback collection adds bounded per-conjunct bookkeeping, emitting one observation tuple per conjunct at query completion. A correction therefore avoids the full-table `ANALYZE` scan a refresh would incur—the intended use case is maintaining optimizer-facing statistics *between* scheduled refreshes, not replacing full refreshes after major schema changes or bulk shifts.

7. Related Work

Feedback-driven statistics and the completion question. Cost-based optimization and selectivity estimation rely on compact statistics built from

scans [1, 2, 3, 4], with streaming summaries such as KLL sketches providing approximate quantiles [23] and query feedback long used to adapt estimates [24]. Self-tuning histograms, STHoles, SASH, and ISOMER refine statistics from observed results [12, 13, 14, 15], and execution-feedback frameworks repair optimizer state over time [25]. OASIS shares this statistics-facing target but treats feedback repair as an *identifiability* problem—a rank condition for when feedback pins the optimizer-relevant statistic (Proposition 1)—and use it to separate two regimes the prior literature does not distinguish: single-column repair, where the maximum-entropy projection is near-optimal and learning is unnecessary, and cross-column repair, where feedback corrects the independence assumption the per-column statistics structurally cannot. The attribute-value-independence assumption and its cost are long documented [4, 8]; engines mitigate it with multi-dimensional or extended statistics built from scans, and STHoles [13] builds multidimensional histograms from feedback. OASIS differs in maintaining both the marginal and, for correlated columns, the joint from query feedback alone—no rescan, with a safety router—and in characterizing when feedback suffices.

Learned cardinality estimation. Learned estimators such as NeuroCard, Naru, DeepDB, FACE, MSCN, FLAT, and Iris predict cardinalities from learned density or representation models [26, 27, 28, 29, 17, 30, 31], and benchmark and deployment studies document both their promise and the difficulty of robustness across workloads and drift [32, 33, 34, 10, 19, 20, 18, 35, 36, 21]. This line remains active in the database-systems literature: recent work learns dynamic sample selectors for cardinality estimation [37] and recursive-network models of complex predicates [38]. These systems output point estimates or estimator state; OASIS instead repairs the optimizer's own marginal *statistics* from query feedback for an existing CBO to consume unchanged, rather than replacing the estimator with a learned model.

Composition, joins, and plan-level learning. Copula and factorized-join estimators such as FactorJoin [22] combine one-dimensional marginals with a dependence or join-key model; our composition-family and FactorJoin experiments embed OASIS as the marginal source in exactly these estimators and show feedback consistency is a prerequisite for safe downstream use. Plan-level and robust optimization systems—mid-query re-optimization, progressive optimization, LEO, Neo, Bao, and Flow-Loss [39, 40, 41, 42, 43, 44, 45, 11]—adjust plan choices or train plan-relevant estimates; OASIS operates below them by correcting the statistics object itself, and the two are complementary.

8. Limitations

The most important boundary is external validity. The cross-column repair is validated on the standard Join Order Benchmark against a live PostgreSQL planner (Section 6.2), but on a sampled conjunctive-predicate suite rather than an end-to-end production query workload with its own statistics-refresh history; the latter remains untested. This boundary is partly structural. To our knowledge no public dataset of production statistics-refresh histories exists, and the self-tuning-histogram line we build on and compare against—STHoles, ISOMER, and QuickSel-H—was likewise established on synthetic or controlled drift rather than on such histories. We therefore adopt the evaluation regime of the prior art and climb a deliberate ladder of increasing realism on top of it: controlled drift families over real data columns; injection into a *real* PostgreSQL optimizer through `pg_statistic`, scored on `EXPLAIN` estimates and plan shapes; and a cross-schema TPC-H check—driven by benchmark-style insert/update/delete refresh streams—that reuses the *same* pretrained prior with no retraining. Each rung is a genuine step beyond the baselines' original validation, though none is yet a production refresh history.

The mechanism is also intrinsically bounded: once feedback pins the statistic, every projection-based method coincides (Proposition 1), so any repair can act only on the part feedback leaves free. For a single column that free part carries no learnable structure beyond the maximum-entropy default (Section 6.1), which is why we do not deploy a learned single-column prior. The exploitable opportunity is the cross-column joint, and even there OASIS repairs only the pairwise correlation that conjunctive feedback observes, not higher-order dependence or join-key skew, for which an external model remains necessary; the cross-column study is also an estimate-quality result, not yet an end-to-end planner injection. The TPC-H check shows only that calibrated statistics reproduce fresh-statistics execution time on one schema and six queries; runtime in general depends on executor behavior, caching, physical design, and cost-model calibration, and a stale mis-estimate can yield a coincidentally faster plan.

Finally, the Router gates on feedback residual, which is a proxy for estimate quality rather than for plan-shape risk: it selects ISOMER on the dense planner configurations, where ISOMER, OASIS, and the Router all sit near the same $\approx 1\%$ new-deviation rate (Table 6), so it does not regress there; but the residual would not by itself flag a plan-shape regression if one

arose. A plan-shape-aware gate is left to future work.

9. Conclusion

OASIS maintains cost-based optimizer statistics from query feedback, with no table rescan and no change to the optimizer. Its organizing finding is a dichotomy. For a single column, feedback nearly determines the marginal: we give an identifiability condition (Proposition 1) for when feedback pins it, and show on held-out *real* datasets that the maximum-entropy projection (the classical ISOMER step) is already near-optimal—a learned prior and the self-tuning histograms STHoles and QuickSel-H give no edge over it. Single-column repair is thus easy and needs no machine learning. The optimizer's larger residual error is the independence assumption on correlated columns, and there feedback helps decisively: a feedback-driven joint repair cuts conjunctive-predicate Q-error by up to 31% (13.6% in aggregate) over independence on real correlated columns, the gain concentrated where the columns are correlated.

Packaged as a statistics-layer middleware and injected into a real PostgreSQL planner, feedback-maintained single-column statistics cut row Q-error from 27.8 to 2.4 and lift fresh-plan match from 56% to 96% without a table scan, and the maintained marginals stay safe for composition and join estimators to consume, guarded by a residual-gated router that never degrades below the maximum-entropy default. OASIS targets the two structural errors in an optimizer's statistics—staleness and independence—using the feedback ordinary queries already produce, with no rescan.

Declarations

Data availability. The drift generators, evaluation harness, and the scripts that reproduce every table and figure in this paper (including the random seeds) are publicly available at `https://github.com/M1zukiqwq/OASIS-OpenSource`; the real datasets used in the evaluation (Adult/Census, Covertype/Forest, Power, Wine Quality, Bike Sharing) are publicly available from the UCI Machine Learning Repository.

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Competing interests. The authors declare no competing interests.

CRediT author contributions. Qichu Tian: Conceptualization, Methodology, Software, Investigation, Data curation, Validation, Visualization, Writing – original draft. **Heng Chen**: Supervision, Conceptualization, Writing – review & editing, Project administration.

Generative AI disclosure. During the preparation of this manuscript the authors used a large language model (Anthropic Claude) to assist with language editing, manuscript restructuring, and the drafting of figure- and table-generating code. No AI tool was used to design the study, to generate or analyze experimental results, or to produce scientific claims. After using this tool the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, T. G. Price, Access path selection in a relational database management system, in: Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data, ACM, 1979, pp. 23–34. doi:10.1145/582095.582099.

[2] S. Chaudhuri, An overview of query optimization in relational systems, in: Proceedings of the 17th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, ACM Press, 1998, pp. 34–43. doi:10.1145/275487.275492.

[3] Y. E. Ioannidis, V. Poosala, Universality of serial histograms, in: Proceedings of the 19th VLDB Conference, 1993, pp. 256–267.

[4] V. Poosala, Y. E. Ioannidis, Selectivity estimation without the attribute value independence assumption, in: Proceedings of the 23rd VLDB Conference, 1997, pp. 486–495.

[5] PostgreSQL Global Development Group, PostgreSQL documentation: Planner statistics, https://www.postgresql.org/docs/current/planner-stats.html, accessed 2026-06-01 (2026).

[6] PostgreSQL Global Development Group, PostgreSQL documentation: Row estimation examples, https://www.postgresql.org/docs/current/row-estimation-examples.html, accessed 2026-06-01 (2026).

[7] PostgreSQL Global Development Group, PostgreSQL documentation: ANALYZE, https://www.postgresql.org/docs/current/sql-analyze.html, accessed 2026-06-01 (2026).

[8] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, T. Neumann, How good are query optimizers, really?, Proceedings of the VLDB Endowment 9 (3) (2015) 204–215.

[9] G. Moerkotte, T. Neumann, G. Steidl, Preventing bad plans by bounding the impact of cardinality estimation errors, Proceedings of the VLDB Endowment 2 (1) (2009) 982–993.

[10] Y. Han, Z. Wu, P. Wu, R. Zhu, J. Yang, L. W. Tan, K. Zeng, G. Cai, Y. Zhang, G. Li, Cardinality estimation in DBMS: A comprehensive benchmark evaluation, Proceedings of the VLDB Endowment 15 (4) (2021) 752–765.

[11] K. Lee, A. Dutt, V. R. Narasayya, S. Chaudhuri, Analyzing the impact of cardinality estimation on execution plans in microsoft SQL server, Proceedings of the VLDB Endowment 16 (11) (2023) 2871–2883. doi:10.14778/3611479.3611494.

[12] A. Aboulnaga, S. Chaudhuri, Self-tuning histograms: Building histograms without looking at data, in: Proceedings of SIGMOD, 1999, pp. 181–192.

[13] N. Bruno, S. Chaudhuri, L. Gravano, STHoles: A multidimensional workload-aware histogram, in: Proceedings of SIGMOD, 2001, pp. 211–222.

[14] L. Lim, M. Wang, J. S. Vitter, SASH: A self-adaptive histogram set for dynamically changing workloads, in: Proceedings of the 29th VLDB Conference, 2003, pp. 369–380. doi:10.1016/B978-012722442-8/50040-9.

[15] V. Markl, N. Megiddo, M. Kutsch, T. M. Tran, P. Lang, V. Raman, Consistent selectivity estimation via maximum entropy, The VLDB Journal 16 (2) (2007) 169–188.

[16] Y. Park, S. Zhong, B. Mozafari, QuickSel: Quick selectivity learning with mixture models, in: Proceedings of SIGMOD, 2020, pp. 1017–1033.

[17] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, A. Kemper, Learned cardinalities: Estimating correlated joins with deep learning, in: CIDR, 2019.

[18] P. Li, W. Wei, R. Zhu, B. Ding, J. Zhou, H. Lu, ALECE: An attention-based learned cardinality estimator for SPJ queries on dynamic workloads, Proceedings of the VLDB Endowment 17 (2) (2023) 197–210. doi:10.14778/3626292.3626302.

[19] B. Li, Y. Lu, S. Kandula, Warper: Efficiently adapting learned cardinality estimators to data and workload drifts, in: Proceedings of SIGMOD, ACM, 2022, pp. 2146–2160. doi:10.1145/3514221.3517914.

[20] P. Negi, Z. Wu, A. Kipf, N. Tatbul, R. Marcus, S. Madden, T. Kraska, M. Alizadeh, Robust query driven cardinality estimation under changing workloads, Proceedings of the VLDB Endowment 16 (6) (2023) 1520–1533. doi:10.14778/3583140.3583164.

[21] Y. Han, H. Wang, L. Chen, Y. Dong, X. Chen, B. Yu, C. Yang, W. Qian, ByteCard: Enhancing ByteDance's data warehouse with learned cardinality estimation, in: Companion of the 2024 International Conference on Management of Data, ACM, 2024, pp. 41–54. doi:10.1145/3626246.3653376.

[22] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, S. Madden, FactorJoin: A new cardinality estimation framework for join queries, Proceedings of the ACM on Management of Data (SIGMOD) 1 (1) (2023) 1–27.

[23] N. Ivkin, J. Nelson, H. Yu, V. Braverman, KLL: Approximately optimal streaming quantile estimation, Proceedings of the ACM on Management of Data 2 (1) (2019) 1–23.

[24] C. M. Chen, N. Roussopoulos, Adaptive selectivity estimation using query feedback, in: Proceedings of the 24th ACM SIGMOD International Conference on Management of Data, ACM Press, 1994, pp. 161–172. doi:10.1145/191839.191874.

[25] S. Chaudhuri, V. R. Narasayya, R. Ramamurthy, A pay-as-you-go framework for query execution feedback, Proceedings of the VLDB Endowment 1 (1) (2008) 1141–1152. doi:10.14778/1453856.1453977.

[26] Z. Yang, A. Kamsetty, S. Luan, E. Liang, Y. Duan, X. Chen, I. Stoica, NeuroCard: One cardinality estimator for all tables, Proceedings of the VLDB Endowment 14 (1) (2020) 61–73.

[27] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, I. Stoica, Deep unsupervised cardinality estimation, Proceedings of the VLDB Endowment 13 (3) (2019) 279–292.

[28] B. Hilprecht, A. Schmidt, M. Kulessa, A. Molina, K. Kersting, C. Binnig, DeepDB: Learn from data, not from queries!, Proceedings of the VLDB Endowment 13 (7) (2020) 992–1005.

[29] J. Wang, C. Chai, J. Liu, G. Li, FACE: A normalizing flow based cardinality estimator, Proceedings of the VLDB Endowment 15 (1) (2021) 72–84. doi:10.14778/3485450.3485458.

[30] R. Zhu, Z. Wu, Y. Han, K. Zeng, A. Pfadler, Z. Qian, J. Zhou, B. Cui, FLAT: Fast, lightweight and accurate method for cardinality estimation, Proceedings of the VLDB Endowment 14 (9) (2021) 1489–1502. doi:10.14778/3461535.3461539.

[31] Y. Lu, S. Kandula, A. C. König, S. Chaudhuri, Pre-training summarization models of structured datasets for cardinality estimation, Proceedings of the VLDB Endowment 15 (3) (2022) 414–426. doi:10.14778/3494124.3494127.

[32] X. Wang, C. Qu, W. Wu, J. Wang, Q. Zhou, Are we ready for learned cardinality estimation?, Proceedings of the VLDB Endowment 14 (9) (2021) 1640–1654. doi:10.14778/3461535.3461552.

[33] J. Sun, J. Zhang, Z. Sun, G. Li, N. Tang, Learned cardinality estimation: A design space exploration and a comparative evaluation, Proceedings of the VLDB Endowment 15 (1) (2021) 85–97. doi:10.14778/3485450.3485459.

[34] K. Kim, J. Jung, I. Seo, W.-S. Han, K. Choi, J. Chong, Learned cardinality estimation: An in-depth study, in: Proceedings of SIGMOD, ACM, 2022, pp. 1214–1227. doi:10.1145/3514221.3526154.

[35] Z. Wang, Q. Zeng, N. Wang, H. Lu, Y. Zhang, CEDA: Learned cardinality estimation with domain adaptation, Proceedings of the VLDB Endowment 16 (12) (2023) 3934–3937. doi:10.14778/3611540.3611589.

[36] R.-A. Wang, Z. Zou, Z. Jing, Cardinality estimation via learned dynamic sample selection, Information Systems 117 (2023) 102252. doi:10.1016/j.is.2023.102252.

[37] R.-A. Wang, Z. Zou, Z. Jing, Cardinality estimation via learned dynamic sample selection, Information Systems 117 (2023) 102252. doi:10.1016/j.is.2023.102252.

[38] Z. Wang, H. Duan, Y. Cheng, G. Min, Learning complex predicates for cardinality estimation using recursive neural networks, Information Systems 124 (2024) 102402. doi:10.1016/j.is.2024.102402.

[39] N. Kabra, D. J. DeWitt, Efficient mid-query re-optimization of sub-optimal query execution plans, in: Proceedings of SIGMOD, 1998, pp. 106–117.

[40] V. Markl, V. Raman, D. E. Simmen, G. M. Lohman, H. Pirahesh, Robust query processing through progressive optimization, in: Proceedings of SIGMOD, ACM, 2004, pp. 659–670. doi:10.1145/1007568.1007642.

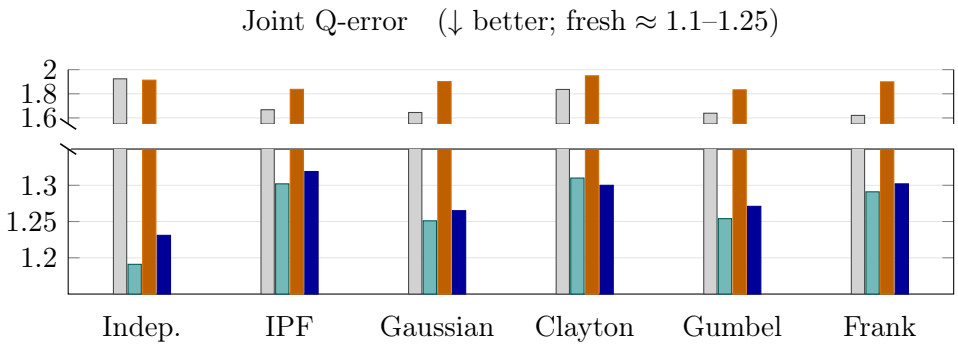
[41] B. Babcock, S. Chaudhuri, Towards a robust query optimizer: A principled and practical approach, in: Proceedings of SIGMOD, ACM, 2005, pp. 119–130. doi:10.1145/1066157.1066172.

[42] M. Stillger, G. Lohman, V. Markl, M. Kandil, LEO – DB2's learning optimizer, in: Proceedings of the 27th VLDB Conference, 2001, pp. 19–28.

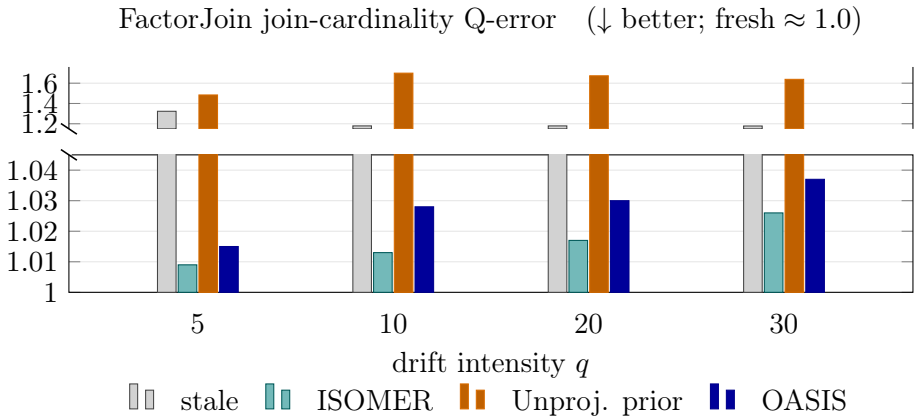
[43] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, N. Tatbul, Neo: A learned query optimizer, Proceedings of the VLDB Endowment 12 (11) (2019) 1705–1718.

[44] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, T. Kraska, Bao: Making learned query optimization practical, in: Proceedings of SIGMOD, 2022, pp. 1275–1288.

[45] P. Negi, R. Marcus, A. Kipf, H. Mao, N. Tatbul, T. Kraska, M. Alizadeh, Flow-loss: Learning cardinality estimates that matter, Proceedings of the VLDB Endowment 14 (11) (2021) 2019–2032. doi:10.14778/3476249.3476259.



(a) Composition family (six estimators; dependence fixed, only the marginal varies): the projection (ISOMER, and OASIS, which coincides with it) recovers most of the stale joint Q-error, whereas the unprojected prior stays near stale.



(b) FactorJoin bin-based join kernel as the join-key distribution drifts (only the single-column key histogram varies). The unprojected prior is *actively harmful*—its Q-error exceeds even stale, because the bilinear kernel amplifies mass-concentration errors—whereas the projected methods track the fresh marginal (≈ 1.0).

Figure 3: Downstream consumption of the maintained statistic. A feedback-consistent (*projected*) statistic is safe for any consumer, whereas an *unprojected* prior is not—weak or negative in the composition family (a), and actively harmful in the bilinear FactorJoin kernel (b). Each panel uses a broken axis (upper for the large values, lower zoomed on the recovered cluster): the feedback projection, not the prior, is what keeps the maintained statistic safe to consume.